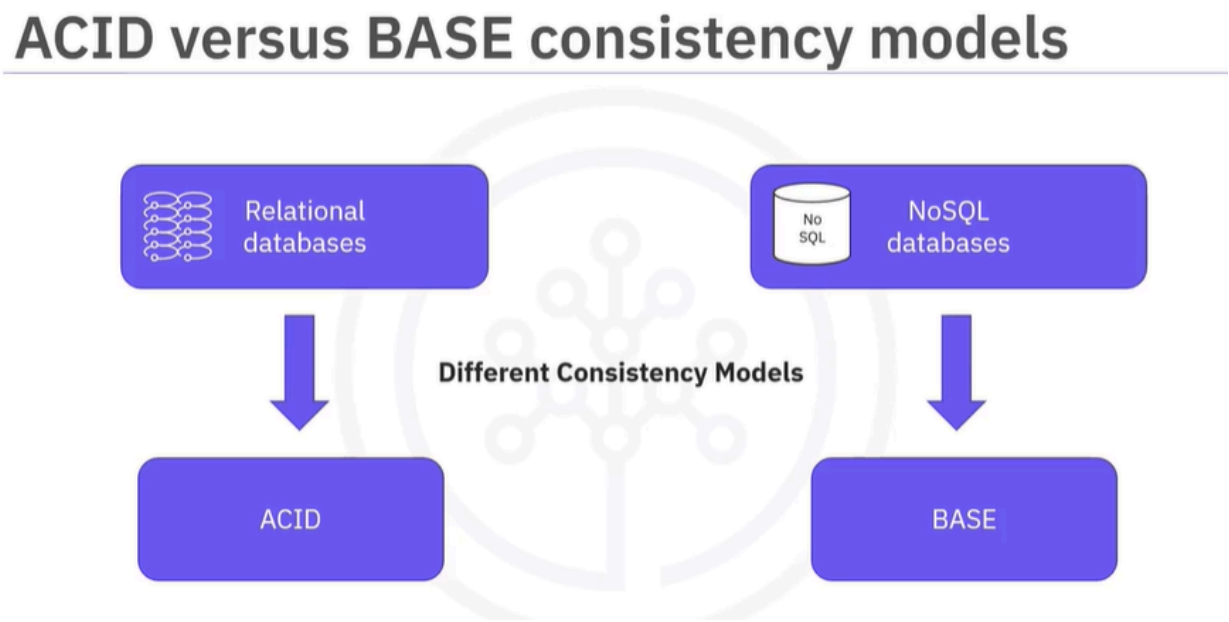
ACID versus BASE Operations

ACID versus BASE Operations: A Simple Explanation

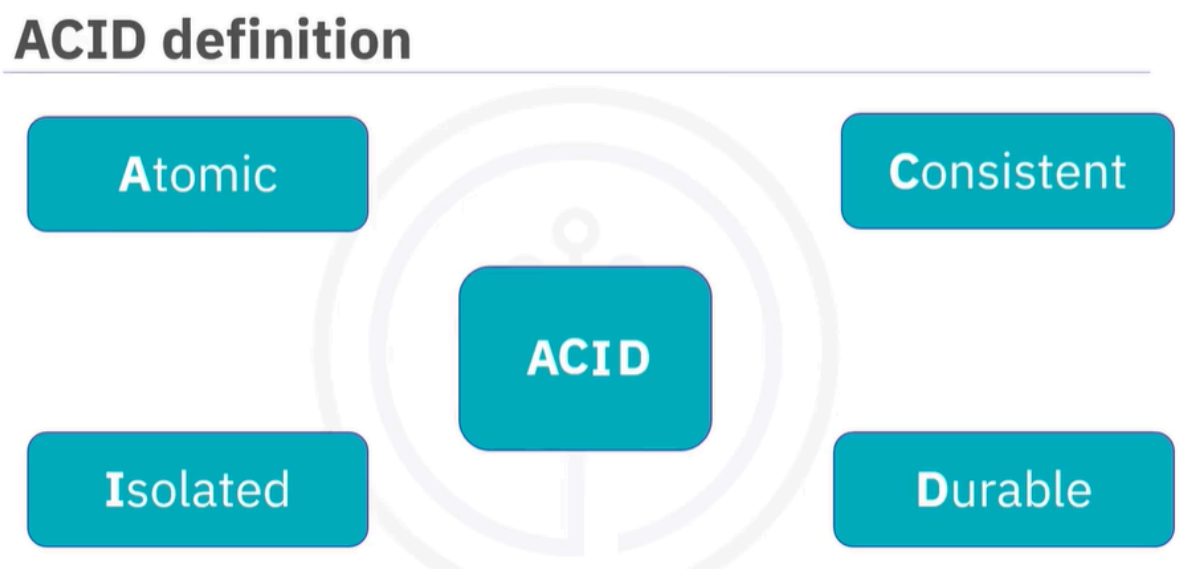
In the world of databases, ACID and BASE are two different ways to ensure that data is handled correctly. ACID stands for Atomicity, Consistency, Isolation, and Durability. Think of it like a bank transaction: when you transfer money, either all parts of the transaction happen successfully (like deducting from one account and adding to another), or none of them do. This ensures that your money is safe and the data remains accurate.

On the other hand, BASE stands for Basically Available, Soft state, and Eventually consistent. Imagine a social media platform where you post a photo. It might take a little time for everyone to see it, but the system is designed to always be available, even if some data isn't perfectly up-to-date right away. This approach is great for applications that need to handle lots of users and data, like Netflix or Spotify, where being available is more important than having every piece of data perfectly synchronized at all times.



- One of the biggest and most striking differences between relational database management systems and NoSQL databases is their data consistency models.

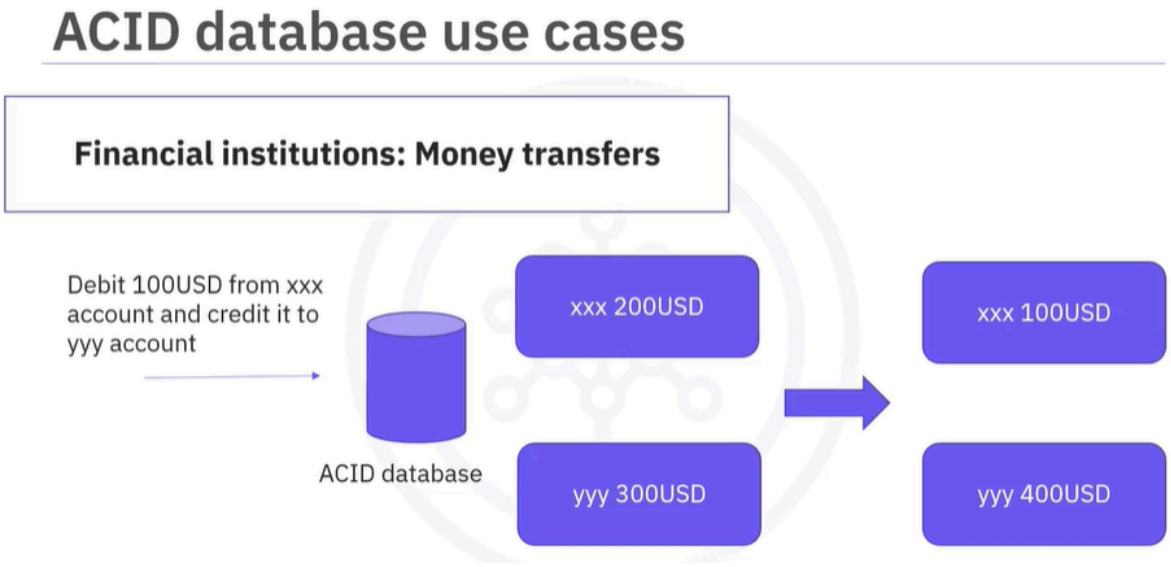
The two most common consistency models that are known and in use nowadays are ACID and BASE. Relational databases use the ACID model, and generally NoSQL databases use the BASE model. For example, MongoDB, which is a document based NoSQL database started supporting ACID transactions from Version 4.0. Although ACID and BASE models are often seen as enemies, or one versus the other, the truth is that both come with their advantages and disadvantages.



ACID consistency model

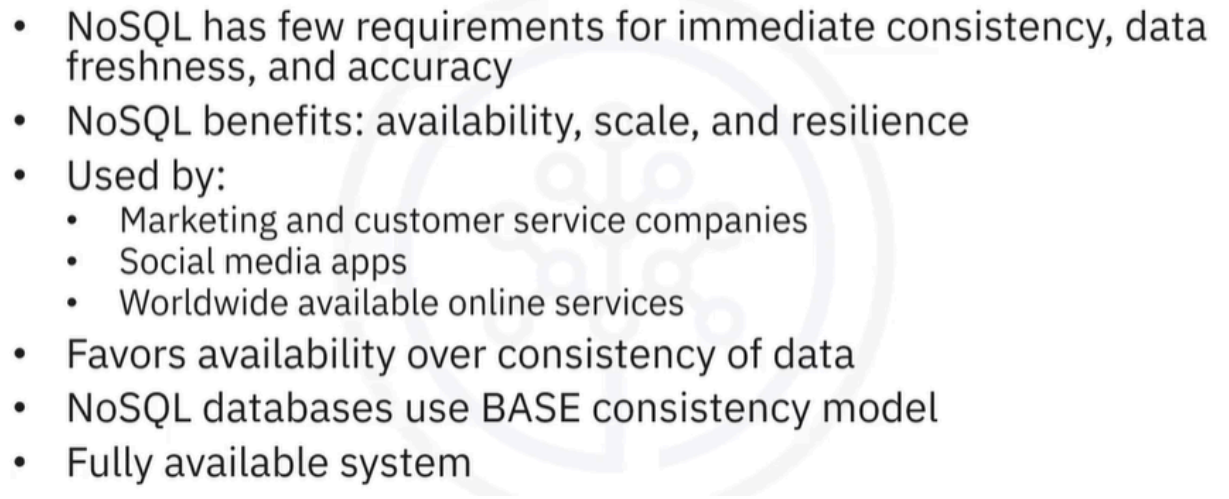
- Used by relational databases
- Ensures a performed transaction is always consistent
- Used by:
  - Financial institutions
  - Data warehousing
- Databases that can handle many small simultaneous transactions => relational databases
- Fully consistent system

- Let's take a closer look at what both terms mean. The ACID acronym stands for atomic. All operations in a transaction succeed, or every operation is rolled back. Consistent. On the completion of a transaction, the structural integrity of the data in the database is not compromised. Isolated. Transactions cannot compromise the integrity of other transactions by interacting with them while they are still in progress. Durable.
- The data related to the completed transaction will persist, even in the case of network or power outages. If a transaction fails, it will not impact the already changed data. Many developers are familiar with ACID transactions from working with relational databases. As such, the ACID consistency model has been the norm for some time. The ACID consistency model ensures that a performed transaction is always consistent.
- This makes the ACID consistency model a good fit for businesses that deal with online transaction processing, such as financial institutions or data warehousing types of applications. These organizations need database systems that can handle many small simultaneous transactions like relational databases can. An ACID system provides a consistent model you can count on for the structural integrity of your data.

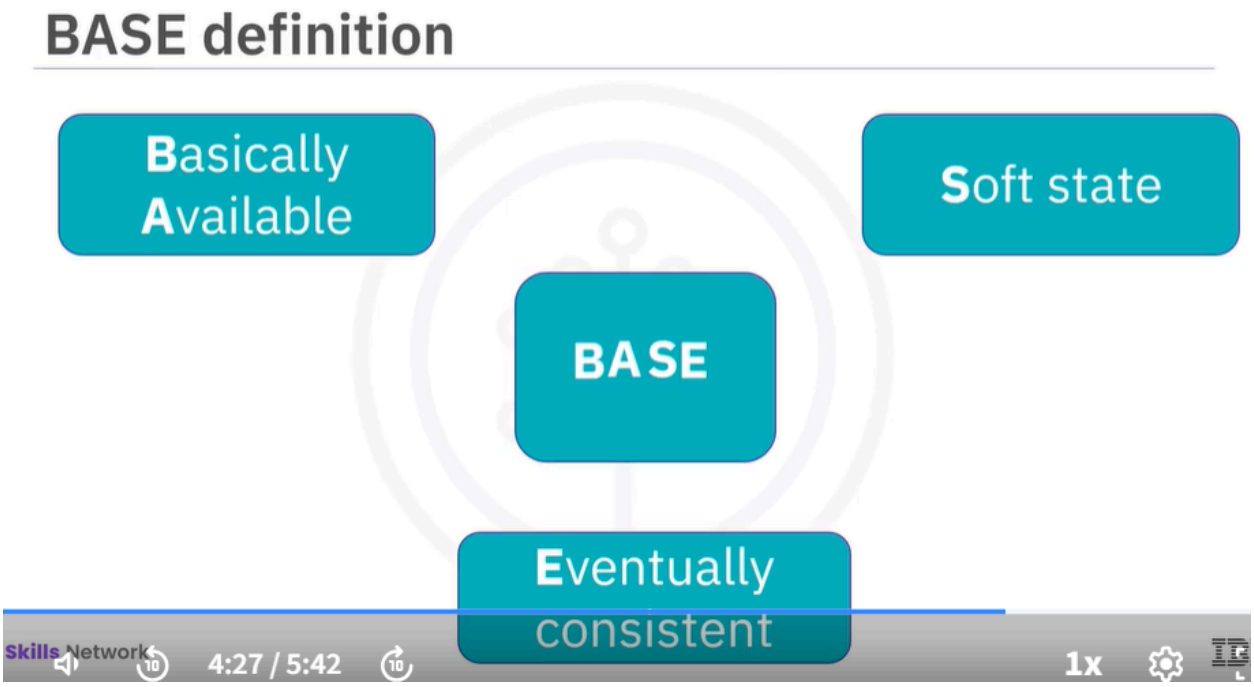


- While there are many use cases for ACID databases, one stands out. Financial institutions will almost exclusively use ACID databases for their money transfers because these operations depend on the atomic nature of ACID transactions. Uninterrupted transaction that is not immediately removed from the database can cause serious complications.

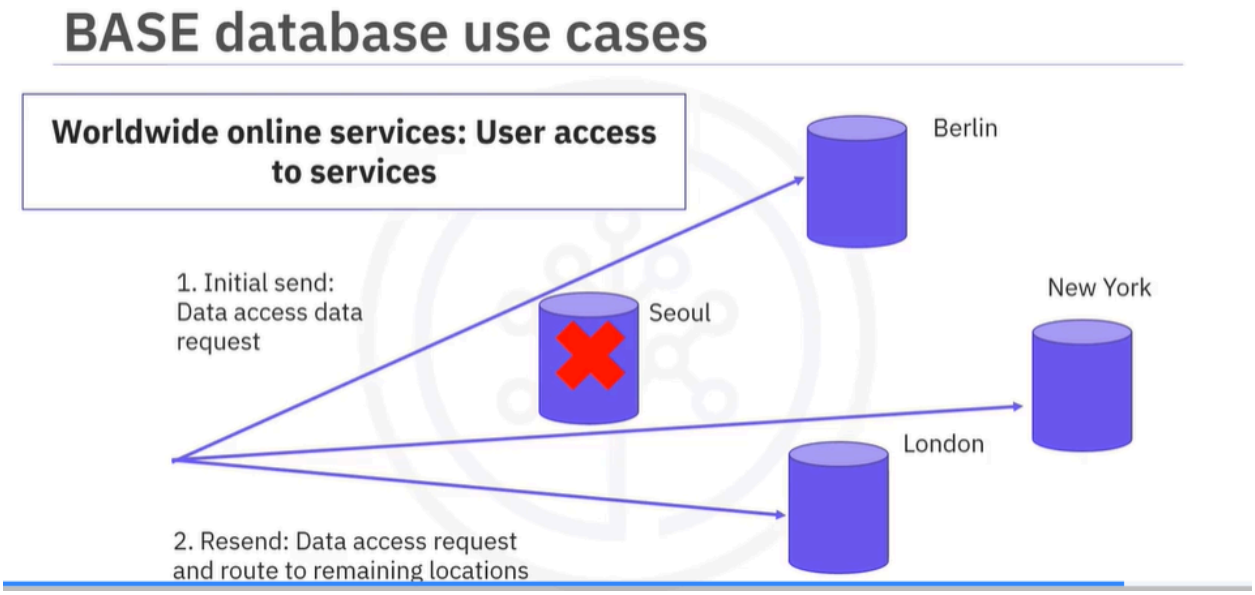
NoSQL and BASE models



- Money could be debited from one account and due to an error never credited to the other, or not credited at all. In the NoSQL database world, ACID transactions are less fashionable because some databases have loosened the requirements for immediate consistency, data freshness, and accuracy. They do this to gain other benefits, such as availability, scale, and resilience. The BASE consistency model is used by marketing and customer service companies that deal with sentiment analysis and social network research.
- Social media applications that contain vast amounts of data and need to be always available, and worldwide online services like Netflix, Spotify, and Uber. The BASE data store values availability over consistency, but it doesn't offer a guaranteed consistency of replicated data at right times. NoSQL databases use the BASE consistency model. Essentially, the BASE consistency model provides high availability.



- The BASE acronym stands for basically available. Rather than enforcing immediate consistency BASE modeled NoSQL databases will ensure the availability of data by spreading and replicating it across the nodes of the database cluster. Soft state. Due to the lack of immediate consistency, data values may change over time. In the BASE model, data stores don't have to be right consistent nor do different replicas have to be mutually consistent all the time. Eventually consistent. The fact that the BASE model does not enforce immediate consistency does not mean that it never achieves it.



- However, until it does, data reads might be inconsistent. BASE data stores can be and are used in specific use cases. The fame of BASE databases increased because well known worldwide online services such as Netflix, Apple, Spotify, and Uber use BASE data stores for applications like user profile data storage. A common characteristic of these services is that they must always be accessible, no matter which part of the world users are located. If a part of their database cluster becomes unavailable, the system needs to be able to serve the user requests without any disruptions.

Recap

In this video, you learned that:

- ACID and BASE are database consistency models
- ACID = Atomicity, Consistency, Isolated, and Durable
- BASE = Basically Available, Soft state, Eventually consistent
- ACID (RDBMS) models focus on consistency
- BASE (NoSQL) models focus on availability
- Usage of each model is based on a case-by-case analysis

What is the main difference between ACID and BASE?

The main difference between ACID and BASE lies in their focus on data consistency and availability:

- **ACID**:
  - **Focuses on Consistency**: Ensures that all transactions are processed reliably and that the database remains in a valid state.
  - **Characteristics**:
    - **Atomicity**: All operations in a transaction succeed or none do.
    - **Consistency**: The database remains in a consistent state after a transaction.
    - **Isolation**: Transactions do not interfere with each other.
    - **Durability**: Once a transaction is committed, it remains so, even in case of failures.

- **BASE**:
  - **Focuses on Availability**: Prioritizes system availability over immediate consistency.
  - **Characteristics**:
    - **Basically Available**: The system is designed to be available most of the time.
    - **Soft State**: The state of the system may change over time, even without new input.
    - **Eventually Consistent**: The system will become consistent over time, but not immediately.

In summary, **ACID** is about ensuring data integrity and consistency, while **BASE** is about ensuring high availability and flexibility, even if it means sacrificing immediate consistency.

How can you apply BASE in a real-world scenario?

Here are two real-world scenarios where the BASE model can be effectively applied:

1. **Social Media Platforms**:
   - **Scenario**: When a user posts a status update or photo, it may not appear instantly for all friends or followers.
   - **Application of BASE**:
     - **Basically Available**: The platform remains accessible, allowing users to post and interact without downtime.
     - **Soft State**: The visibility of the post may change over time as it propagates through the network.
     - **Eventually Consistent**: Eventually, all users will see the update, but there may be a delay, ensuring that the system prioritizes availability over immediate consistency.

2. **E-commerce Websites**:
    - **Scenario**: When a customer places an order, the inventory may not update immediately across all systems.
    - **Application of BASE**:
        - **Basically Available**: Customers can browse products and place orders even if some inventory data is temporarily outdated.
        - **Soft State**: The inventory levels may fluctuate as orders are processed and updated.
        - **Eventually Consistent**: The inventory will be updated to reflect the correct stock levels, but there may be a short period where the displayed stock is not accurate.

These examples illustrate how BASE can be beneficial in environments where high availability and user experience are critical, even if it means that data consistency is not immediate.

What challenges might arise when using BASE over ACID?

When using BASE over ACID, several challenges may arise:

1. **Data Inconsistency**:
    - Since BASE prioritizes availability over immediate consistency, there may be times when different parts of the system show different data. This can lead to confusion for users who expect real-time accuracy.
2. **Eventual Consistency**:
    - While BASE systems aim for eventual consistency, there can be delays in synchronizing data across nodes. This means that users might see outdated information for a period, which can be problematic in time-sensitive applications.
3. **Complexity in Application Logic**:

- Developers may need to implement additional logic to handle scenarios where data is not immediately consistent. This can complicate application design and increase the potential for bugs.

4. **User Experience Issues**:
    - Users may experience situations where actions they take (like placing an order) do not reflect immediately in the system, leading to frustration or confusion.
5. **Data Integrity Risks**:
    - In systems where data integrity is critical (like financial transactions), relying on BASE can pose risks, as the lack of immediate consistency may lead to errors or discrepancies in data.
6. **Difficulty in Debugging**:
    - Troubleshooting issues in a BASE system can be more challenging due to the asynchronous nature of data updates, making it harder to pinpoint where inconsistencies arise.

These challenges highlight the importance of carefully considering the use case and requirements of an application when choosing between BASE and ACID models.

What is the main difference between ACID and BASE?

The main difference between ACID and BASE lies in their approach to data consistency and availability:

- **ACID**:
    - **Focus on Consistency**: Ensures that all transactions are processed reliably, maintaining the integrity of the database.
    - **Key Characteristics**:
        - **Atomicity**: All operations in a transaction succeed or none do.

        - **Consistency**: The database remains in a valid state after a transaction.
        - **Isolation**: Transactions do not interfere with each other.
        - **Durability**: Once a transaction is committed, it remains so, even in case of failures.
- **BASE**:
    - **Focus on Availability**: Prioritizes system availability over immediate consistency.
    - **Key Characteristics**:
        - **Basically Available**: The system is designed to be available most of the time.
        - **Soft State**: The state of the system may change over time, even without new input.
        - **Eventually Consistent**: The system will become consistent over time, but not immediately.

In summary, **ACID** emphasizes data integrity and consistency, while **BASE** emphasizes high availability and flexibility, accepting that data may not be immediately consistent.